We will only need to use the `x_pos` key. In addition to getting the state after calling the `step` method, you can also retrieve the state at any point by calling `env.render ("rgb_array")`. That's basically all you need to know about the environment in order to train an agent to play it.

8.4. Preprocessing and the Q-network

The raw state is an RGB video frame with dimensions (240, 256, 3), which is unnecessarily high-dimensional and would be computationally costly for no advantage. We will convert these RGB states into grayscale and resize them to 42 × 42 to allow our model to train much faster.

Listing 8.2. Downsample state and convert to grayscale

```
import matplotlib.pyplot as plt
from skimage.transform import resize
1
import numpy as np

def downscale_obs(obs, new_size=(42,42), to_gray=True):
    if to_gray:
        return resize(obs, new_size,
anti_aliasing=True).max(axis=2)   2
    else:
        return resize(obs, new_size, anti_aliasing=True)
```

- ***1*** The scikit-image library has an image-resizing function built in.
- ***2*** To convert to grayscale, we simply take the maximum values across the channel dimension for good contrast.

The `downscale_obs` function accepts the state array (`obs`), a tuple indicating the new size in height and width, and a Boolean for whether to convert to grayscale or not. We set it to `True` by default since that is what we want. We use the scikit-image library's `resize` function, so you may need to install it if you don't have it already (go to the download page at https://scikit-image.org/). It's a very useful library for working with image data in the form of multidimensional arrays.

You can use matplotlib to visualize a frame of the state: